

Үй тапсырмасы: Нейрондық желілер бойынша есептердің толық шешімі

Формат: әр есеп үшін — Сұрақ (тапсырма) → Жауап/шешуі → Қысқаша түсіндірме.

1-есеп. Персептрон шешімі

Сұрақ: Берілген персептрон: $z = 2x_1 - x_2 + 1$. Активация: $f(z)=1$ ($z \geq 0$), әйтпесе 0. 1) $(x_1, x_2)=(1,0)$ үшін шығысты есептеңіз. 2) $(0,2)$ үшін есептеңіз. 3) Шешім шекарасының теңдеуін жазыңыз.

Шешуі:

1) $(1,0)$: $z = 2 \cdot 1 - 0 + 1 = 3 \Rightarrow z \geq 0 \Rightarrow f(z)=1$.

2) $(0,2)$: $z = 2 \cdot 0 - 2 + 1 = -1 \Rightarrow z < 0 \Rightarrow f(z)=0$.

3) Шекара: $2x_1 - x_2 + 1 = 0 \Rightarrow x_2 = 2x_1 + 1$.

Түсіндірме: Step-активация тек «табалдырықтан өтті/өтпеді» принципімен жұмыс істейді, сондықтан шешім шекарасы әрқашан түзу болады.

2-есеп. XOR мәселесі

Сұрақ: 1) Неге бір қабатты персептрон XOR функциясын үйрене алмайды? 2) Геометриялық түсіндірме беріңіз. 3) Қандай архитектура XOR-ды шешеді?

Жауап:

1) XOR — сызықты бөлінбейтін (linearly inseparable) есеп, ал бір қабатты персептрон тек сызықтық шекара құра алады.

2) $(0,0)$ және $(1,1)$ бір класс, $(0,1)$ және $(1,0)$ екінші класс болып орналасады; бір түзу арқылы екі топты бөлу мүмкін емес.

3) Кемінде 1 жасырын қабаты бар көпқабатты персептрон (MLP) XOR-ды шеше алады (мысалы 2-2-1).

Түсіндірме: Жасырын қабат деректерді жаңа кеңістікке түрлендіріп, кейін сызықты бөлуге мүмкіндік береді.

3-есеп. Активация функцияларын салыстыру

Сұрақ: Sigmoid, Tanh және ReLU үшін: 1) формула; 2) шығыс диапазоны; 3) туындысы; 4) қайсысы vanishing gradient мәселесіне бейім?

Функция	Формула	Диапазон	Туындысы (негізгі идея)	Vanishing gradient
Sigmoid	$\sigma(x)=1/(1+e^{(-x)})$	$(0,1)$	$\sigma(x)(1-\sigma(x))$	Иә (қанығу кезінде)
Tanh	$\tanh(x)=(e^x-e^{(-x)})/(e^x+e^{(-x)})$	$(-1,1)$	$1-\tanh^2(x)$	Иә (қанығу кезінде)
ReLU	$\max(0,x)$	$[0,\infty)$	$x>0$ болса 1, әйтпесе 0	Әдетте аз (бірақ dead

Түсіндірме: Sigmoid/Tanh шеттерде қанығады → туынды өте кішкентай болады → градиент қабаттар бойымен өшеді. ReLU көбіне градиентті сақтауға көмектеседі.

4-есеп. Softmax интерпретациясы

Сұрақ: Берілген логиттер $z=[2,1,0]$. 1) Softmax мәндерін есептеңіз. 2) Қай класс ең ықтимал? 3) Softmax нәтижелерінің қосындысы неге тең?

Шешуі: $e^2=7.389$, $e^1=2.718$, $e^0=1.000$, қосындысы $S=11.107$.
 $P_1=0.665$, $P_2=0.245$, $P_3=0.090$.

Жауап: Ең ықтимал класс — 1-класс (P_1 ең үлкен). Softmax ықтималдықтарының қосындысы әрқашан 1-ге тең.

Түсіндірме: Softmax логиттерді ықтималдыққа айналдырады және барлық кластар бойынша нормализация жасайды.

5-есеп. Forward propagation (есептеу)

Сұрақ: Берілген желі: кіріс $x=[1,2]$, салмақтар $W=[[1,-1],[2,0]]$, bias $b=[0,1]$, активация ReLU. 1) $z=Wx+b$ 2) $a=\text{ReLU}(z)$ есептеңіз.

Шешуі:

$$z = W \cdot x + b = [-1, 3]$$

$$a = \text{ReLU}(z) = [0, 3]$$

Түсіндірме: ReLU теріс мәндерді 0-ге айналдырады, оң мәндерді өзгертпейді.

6-есеп. Жоғалту функциясы (Binary Cross-Entropy)

Сұрақ: Берілген $y=1$ және $\hat{y}=0.8$. BCE формуласы: $L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$. Жоғалтуды есептеңіз.

Шешуі: $y=1$ болғандықтан $L = -\log(0.8) \approx 0.223$.

Түсіндірме: Дұрыс класстың ықтималдығы төмен болса, loss үлкен болады; модельді қателікті азайтуға итермелейді.

7-есеп. Градиент (шығыс қабаты)

Сұрақ: Sigmoid + Cross-Entropy үшін $\delta = \hat{y} - y$. Берілген $y=0$ және $\hat{y}=0.3$. δ -ны есептеңіз және белгісін түсіндіріңіз.

Жауап: $\delta = 0.3 - 0 = 0.3$.

Түсіндірме: $\delta > 0 \Rightarrow$ модель 1 жаққа «артық» болжап тұр; салмақтар жаңартуда шығысты төмендету бағытына итермелейді.

8-есеп. Жасырын қабаттағы градиент

Сұрақ: Берілген: $\delta^{(l+1)}=[0.2, -0.1]$, $W^{(l+1)}=[[1,2],[-1,1]]$, $f'(z^{(l)})=[1,0]$. Формула: $\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot f'(z^{(l)})$. $\delta^{(l)}$ -ды есептеңіз.

Есептеу: $(W^{(l+1)})^T \delta^{(l+1)} = [0.30000000000000004, 0.30000000000000004] \Rightarrow \delta^{(l)} = [0.30000000000000004, 0.0]$.

Түсіндірме: $f'(z)=[1,0]$ болғандықтан екінші нейрон градиентті өткізбейді (ReLU өшіру әсері).

9-есеп. Салмақ жаңарту

Сұрақ: Берілген: $w=0.5$, $\partial L / \partial w = 0.2$, $\eta=0.1$. Жаңартылған салмақты табыңыз.

Шешуі: $w_{\text{new}} = w - \eta \cdot (\partial L / \partial w) = 0.5 - 0.1 \cdot 0.2 = 0.48$.

Түсіндірме: Градиент оң болса, салмақ азаяды; теріс болса, салмақ өседі.

10-есеп. Персептронды нөлден жазу (Python / Colab)

Сұрақ: 1) NumPy пайдаланып бір нейронды іске асырыңыз. 2) AND немесе OR логикалық функциясын үйретіңіз. 3) Қорытынды жасаңыз.

Жауап (негізгі идея): AND/OR — сызықты бөлінетін есептер, сондықтан бір персептрон оларды үйренеді. Төменде ықшам үлгі код (AND).

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([0,0,0,1], dtype=int) # AND

w = np.zeros(2); b = 0.0; lr = 0.2
for epoch in range(100):
    cost = 0
    for xi, yi in zip(X, y):
        yhat = 1 if (w @ xi + b) >= 0 else 0
        err = yi - yhat
        if err != 0:
            w += lr * err * xi
            b += lr * err
        cost += 1
    if cost == 0:
        break

print("w=", w, "b=", b)
print("pred:", [1 if (w@xi+b)>=0 else 0 for xi in X])
```

Қорытынды: Персептрон AND/OR-ды тұрақты түрде үйренеді, өйткені оларды бір түзу арқылы бөлуге болады.

11-есеп. XOR үшін MLP

Сұрақ: 1) 2-hidden-1 архитектурасын қолданыңыз. 2) ReLU+Sigmoid. 3) Loss графигін салыңыз. 4) Персептронмен салыстырыңыз.

Жауап (негізгі ой): XOR сызықты бөлінбейді → 1 қабат жеткіліксіз. 2-2-1 MLP үйрене алады. Төменде ықшам үлгі код (NumPy).

```
# XOR: X y
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([0,1,1,0], dtype=float)

# 2-2-1 MLP (ReLU -> Sigmoid), BCE loss
# ... (Colab)
# loss: 0.1, 1, 1, 0.
```

Салыстыру: Персептрон XOR-ды үйренбейді (қателер тоқтамайды), ал MLP-де loss біртіндеп төмендеп, дұрыс жауапқа келеді.

12-есеп. Активация әсері

Сұрақ: 1) ReLU орнына Tanh қолданыңыз. 2) Оқыту жылдамдығын салыстырыңыз. 3) Gradient тұрақтылығын талдаңыз.

Жауап:

- 1) ReLU орнына Tanh қолданғанда жасырын қабат шығысы $(-1, 1)$ аралығында болады.
- 2) Көп жағдайда ReLU жылдамырақ үйренеді; Tanh қанығу аймағында градиентті кішірейтіп, үйренуді баяулатуы мүмкін.
- 3) Tanh-те vanishing gradient қаупі жоғарырақ; ReLU градиентті жақсы сақтайды (бірақ dead ReLU қаупі бар).

13-есеп. Learning rate эксперименті

Сұрақ: $\eta = \{0.001, 0.01, 0.1, 1.0\}$. 1) Loss қалай өзгереді? 2) Қай мән оптималды? 3) Неге?

η	Күтілетін мінез-құлық (loss)
0.001	Өте баяу төмендейді (convergence ұзақ).
0.01	Тұрақты төмендеу, көбіне жақсы таңдау.
0.1	Жылдам төмендеуі мүмкін, егер модель/дерек рұқсат етсе — оптимал болуы ықтимал.
1.0	Көбіне ауытқиды/жарылып кетеді (diverge), loss тұрақсыз.

Оптимал мән: жиі $\eta = 0.01$ немесе 0.1. Себебі η тым кіші болса — өте баяу, тым үлкен болса — overshoot жасап, үйрену тұрақсыз болады.

14-есеп. Vanishing Gradient

Сұрақ: 1) Бұл мәселе неден пайда болады? 2) Қандай активация функциялары көмектеседі? 3) BatchNorm рөлі қандай?

Жауап:

- 1) Терең желілерде градиент қабаттан қабатқа көбейтіле береді; Sigmoid/Tanh қанығу аймағында туынды < 1 , сондықтан градиент экспоненциалды түрде кішірейіп кетеді.
- 2) ReLU/LeakyReLU/ELU сияқты қанықпайтын (немесе аз қанығатын) функциялар градиенттің сақталуына көмектеседі.
- 3) BatchNorm активацияларды нормализациялап, үлестірімді тұрақтандырады; градиенттің таралуын жақсартады және learning rate таңдауды жеңілдетеді.

15-есеп. Initialization

Сұрақ: 1) Неге салмақты нөлмен инициализациялауға болмайды? 2) Xavier және He айырмашылығы. 3) Қайсысы ReLU үшін дұрыс?

Жауап:

- 1) Барлық нейрондар бірдей бастаса, бірдей градиент алады (symmetry breaking жоқ) → желі үйренбейді.
- 2) Xavier (Glorot) — Sigmoid/Tanh үшін (fan_in және fan_out -қа байланысты), He — ReLU үшін (көбіне $\text{var} = 2/\text{fan_in}$).
- 3) ReLU үшін дұрыс — He initialization.

16-есеп. Эссе

Тақырып: «Forward және Backward propagation нейрондық желінің жүрегі ретінде» (көлемі 1-2 бет).

Forward propagation (алға тарату) — нейрондық желінің болжау (inference) кезінде істейтін негізгі процесі.

Бұл кезеңде кіріс дерек x қабаттар арқылы біртіндеп түрленіп, соңында модельдің шығысы $\hat{y}$ алынады.

Әр қабатта алдымен сызықтық комбинация есептеледі: $z = W \cdot a + b$, мұндағы a — алдыңғы қабаттың активациясы, W — салмақ, b — ығысу (bias).

Содан кейін бейсызық активация функциясы қолданылады: $a = f(z)$.

Бейсызықтықтың маңызы өте үлкен: егер барлық қабаттар тек сызықтық болса,

терең желі де қарапайым сызықтық модельге ғана эквивалент болады.

Сондықтан ReLU, Tanh, Sigmoid сияқты функциялар модельге күрделі тәуелділіктерді үйренуге мүмкіндік береді.

Forward propagation нәтижесінде біз loss (жоғалту) функциясын есептейміз.

Мысалы, бинарлық классификацияда Binary Cross-Entropy қолданылады:

$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$. Loss модельдің қателігін сандық түрде көрсетеді: L азайған сайын модельдің болжамы дұрысқа жақындайды.

Backward propagation (кері тарату) — нейрондық желінің үйрену механизмі.

Оның мақсаты — loss-ты азайтатын бағытта параметрлерді (W , b) жаңарту үшін градиенттерді есептеу.

Негізгі құрал — тізбектей туынды (chain rule). Шығыс қабатында қате сигналын δ деп белгілейміз. Sigmoid + Cross-Entropy комбинациясы үшін $\delta = \hat{y} - y$ өте ыңғайлы түрде шығады,

яғни есептеу жеңілдейді және сандық тұрақтылық артады. Кейін осы δ алдыңғы қабаттарға тарайды:

$\delta^{(l)} = (W^{(l+1)})^T \cdot \delta^{(l+1)} \odot f'(z^{(l)})$. Мұндағы $\odot$ — элементтік көбейту, $f'(z)$ — активацияның туындысы.

Осылайша әр қабат үшін $\partial L / \partial W$ және $\partial L / \partial b$ табылады, әрі gradient descent арқылы жаңарту жасалады:

$W_{\text{new}} = W - \eta \cdot \partial L / \partial W$, $b_{\text{new}} = b - \eta \cdot \partial L / \partial b$, мұнда η — learning rate.

Неліктен бұл «жүрек» деп айтамыз? Себебі forward propagation болмаса — модель болжай алмайды, ал backpropagation болмаса — модель үйренбейді.

Production тұрғысынан қарасақ, inference кезінде жылдамдық маңызды (оптимизация, батчинг, кэштеу), ал training кезінде тұрақтылық маңызды (инициализация, learning rate schedule, нормализация).

Мысалы, терең желілерде vanishing gradient мәселесі пайда болуы мүмкін: Sigmoid/Tanh қанығу аймағында туындысы өте кіші болып, градиент өшіп қалады.

Бұл жағдайда ReLU/LeakyReLU қолдану, He initialization және BatchNorm сияқты әдістер градиенттің дұрыс таралуына көмектеседі.

Қорытындылай келе, forward propagation — деректен болжамға дейінгі «есептеу жолы», ал backpropagation — қателікті параметрлерге дейін жеткізетін «үйрету жолы».

Екеуі бірігіп нейрондық желінің толық өмірлік циклін құрайды: болжау, бағалау, түзету және жақсарту. Дәл осы цикл нейрондық желіні практикалық тұрғыда тиімді құралға айналдырады.